

Grundlagenpraktikum: Rechnerarchitektur**Instruktionsliste x86-64 (bis SSE4.2)**

Im Folgenden sind nahezu alle Instruktionen der x86-64-Architektur einschließlich der Erweiterungen bis SSE3/SSSE3/SSE4.1/SSE4.2 aufgelistet. Die x86-64-Architektur hat zahlreiche weitere Erweiterungen erhalten, die zwar weit verbreitet sind aber hier nicht separat aufgelistet werden.

† Gekennzeichnete Instruktionen haben eine hohe Latenz, benötigen Microcode, oder sind aus anderen Gründen performanztechnisch nicht empfehlenswert.

* Gekennzeichnete Instruktionen gehört zu einer der Erweiterungen SSE3, SSSE3, SSE4.1 oder SSE4.2.

Instruktionen auf General-Purpose Registern**Arithmetisch**

- add, adc — Add (with Carry)
- cmp — Compare
- dec — Decrement by 1
- div[†] — Unsigned Divide
- idiv[†] — Signed Divide
- imul — (Signed) Multiply
- inc — Increment by 1
- mul — Unsigned Multiply
- neg — Two's Complement Negation
- sub, sbb — Subtract (with Carry)
- xadd[†] — Exchange and Add

Logisch

- and — Logical AND
- bsf, bsr — Bit Scan Forward/Reverse
- not — One's Complement Negation
- or — Logical OR
- rcl[†], rcr[†] — Rotate with Carry
- rol, ror — Rotate
- shl, shr, sar — Shift
- shld, shrd — Double Precision Shift
- test — Logical Compare
- xor — Logical Exclusive OR

String-Instruktionen

- cld, std — Clear/Set Direction Flag
- cmps — Compare String Operands
- lods — Load String
- movs — Move Data From String to String
- scas — Scan String
- stos — Store String

Kontrollfluss

- call — Call Procedure
- jmp — Jump
- jcc — Jump if Condition is met

- jcxz[†] — Jump if RCX is zero
- loop[†], loopcc[†] — Loop According to ECX
- ret — Return from Procedure

Sonstige

- bswap[†] — Byte Swap
- bt, btc, btr, bts — Bit Test (and Complement/Reset/Set)
- cbw, cwde, cdqe — Extend byte/word/-doubleword
- cwd, cdq, cqo — Extend word/doubleword/quadword
- clc[†], cmc[†], stc[†] — Clear/Compl./Set Carry Flag
- cmovcc — Conditional Move
- cmpxchg[†], cmpxchg16b[†] — Compare and Exchange
- cpuid[†] — CPU Identification
- enter[†] — Make Stack Frame
- lahf, sahf — Load Flags into AH/Store AH into Flags
- lea — Load Effective Address
- leave — High Level Procedure Exit
- mov — Move
- movbe[†] — Move After Swapping Bytes
- movsx — Move with Sign-Extension
- movzx — Move with Zero-Extension
- nop — No Operation
- push, pop — Push/Pop to/from the Stack
- pushfq[†], popfq[†] — Push/Pop rflags
- setcc — Set Byte on Condition
- syscall — Fast System Call
- xchg[†] — Exchange
- xlat[†] — Table Look-up Translation

SSE-Instruktionen

Move

- movss, movsd
- movaps, movapd, movdqa
- movups, movupd, movdqu
- movlps, movhps
- movlpd, movhpd
- movntps, movntpd, movntdq, movntdqa*
- movd, movq
- maskmovdqu
- lddqu*

Shuffle

- blendps*, blendpd*, pblendw*
- blendvps*, blendvpd*, pblendvb*
- movddup*, movsldup*, movshdup*
- movlhps, movhlps
- pshufb*
- pshufhw, pshufw
- shufps, shufpd, pshufd
- palignr*

Floating-Point Arithmetik

- rcpss, rcpps
- rsqrtss, rsqrtps

Integer Arithmetik

- phminposuw*
- pmaddubsw*
- pmaddwd

- pmovmskb, movmskps, movmskpd
- pmovsx*, pmovzx*
- pmulhrsw*
- psadbw
- pslldq, psrldq
- mpsadbw
- ptest*
- pcmpestri[†], pcmpestrm[†]
- pcmpistri[†], pcmpistrm[†]
- packsswb, packssdw
- packuswb, packusdw*

Conversions

	Scalar	Vector
Int→F32	cvtsi2ss	cvtdq2ps
Int→F64	cvtsi2sd	cvtdq2pd
F64→F32	cvtsd2ss	cvtpd2ps
F32→F64	cvtss2sd	cvtps2pd
F32→Int	cvtss2si	cvtps2dq
–Trunc.	cvtts2si	cvttps2dq
F64→Int	cvtsd2si	cvtpd2dq
–Trunc.	cvttsd2si	cvttpd2dq

Sonstige

- ldmxcsr, stmxcsr

	Skalar F32/F64	Vektor F32/F64	V-SI8	V-UI8	V-SI16	V-UI16	V-SI32	V-UI32	V-SI64	V-UI64
Add	addss/addsd	addps/addpd	paddb		paddw		paddd		paddq	
–Saturating	–	–	paddsb	paddusb	paddsw	paddusw	–	–	–	–
Horz. Add	–	haddps*/haddpd*	–	–	phaddw*		phaddd*		–	–
–Saturating	–	–	–	–	phaddsw*		–	–	–	–
Sub	subss/subsd	subps/subpd	psubb		psubw		psubd		psubq	
–Saturating	–	–	psubsb	psubusb	psubsw	psubusw	–	–	–	–
Horz. Sub	–	hsubps*/hsubpd*	–	–	phsubw*		phsubd*		–	–
–Saturating	–	–	–	–	phsubsw*		–	–	–	–
Add/Sub	–	addsubps/addsubpd	–	–	–	–	–	–	–	–
Multiply	mulss/mulsd	mulps/mulpd	–	–	pmullw		pmulld*		–	–
					pmulhw	pmulhuw	pmuldq*	pmuludq		
Divide	divss ⁺ /divsd ⁺	divps ⁺ /divpd ⁺	–	–	–	–	–	–	–	–
Square Root	sqrtps ⁺ /sqrtsd ⁺	sqrtps ⁺ /sqrtpd ⁺	–	–	–	–	–	–	–	–
Dot Product	–	dpps ⁺⁺ /dppd ⁺⁺	–	–	–	–	–	–	–	–
Compare	cmpss/cmpps	cmpps/cmppd	pcmpeqb		pcmpeqw		pcmpeqd		pcmpeqq*	
	ucomiss/ucomisd		pcmpgtb	–	pcmpgtw	–	pcmpgtd	–	pcmpgtq	–
Maximum	maxss/maxsd	maxps/maxpd	pmaxsb*	pmaxub	pmaxsw	pmaxuw*	pmaxsd*	pmaxud*	–	–
Minimum	minss/minsd	minps/minpd	pminsb*	pminub	pminsw	pminuw*	pminsd*	pminud*	–	–
Round	roundss ⁺ /roundsd ⁺	roundps ⁺ /roundpd ⁺	–	–	–	–	–	–	–	–
Absolute	–	–	pabsb*	–	pabsw*	–	pabsd*	–	–	–
Average	–	–	–	pavgb	–	pavgw	–	–	–	–
Sign	–	–	–	psignb*	–	psignw*	–	psignd*	–	–
Shift Left	–	–	–	–	psllw		pslld		psllq	
Shift Right	–	–	–	–	psraw	psrlw	psrad	psrld	psraq	psrlq
And	–	andps/andpd	pand							
And Not	–	andnps/andnpd	pandn							
Or	–	orps/orpd	por							
Xor	–	xorps/xorpd	pxor							
Unpack low	–	unpcklps/unpcklpd	punpcklbw		punpcklwd		punpckldq		punpckldq	
Unpack high	–	unpckhps/unpckhpd	punpckhbw		punpckhwd		punpckhdq		punpckhdq	
Extract	–	extractps/–	pextrb*		pextrw*		pextrd*		pextrq*	
Insert	–	insertps/–	pinsrb*		pinsrw		pinsrd*		pinsrq*	